

Statistical & Numerical Methods Final Assignment

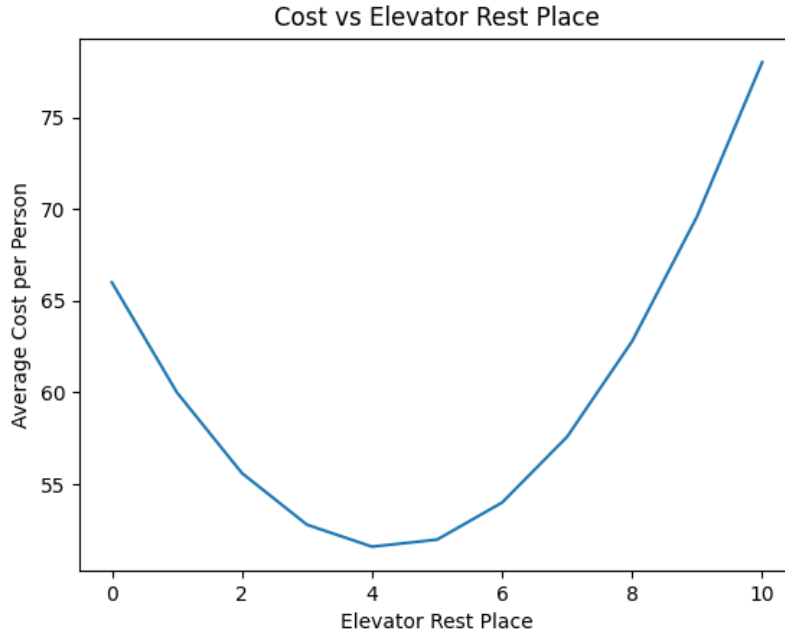
Daniel Haim Breger, 316136944

Technion

(Date: July 17, 2026)

1 Lift Waiting Location

The code solving this, with documentation, is attached as q1.py, the result it produces is floor 4.



1.1 Bonus Calculation

To find the optimal resting floor we'll start by analyzing the paths it takes, focusing on a single person. Through all the assumptions we can simplify the model to:

1. Person enters on floor 0.
2. Person go to floor X.
3. Person then goes to 3 random floors.
4. From the final floor he reaches, the person goes down to 0 and leaves.

in each movement step the elevator performs

1. Elevator goes to origin floor from its resting point.
2. Elevator goes to destination floor from the origin floor.
3. Elevator goes to the resting point from the destination floor.

Each ride's cost calculation is

$$C = |rest - origin| + |dest - origin| + |rest - dest| \quad (1)$$

we can notice that the middle element doesn't depend on the rest point so we can simply ignore it. Therefore the goal we want to minimize is

$$C = E(|rest - origin|) + E(|rest - dest|) \quad (2)$$

but from the symmetry of a sufficiently large random selection of people and trips we can simplify this to

$$C = 2E(|rest - X|) \quad (3)$$

where X is a floor. To this we add the trips as the mapping

$$\{0, f_1, f_2, f_3, f_4\} \rightarrow \{f_1, f_2, f_3, f_4, 0\} \quad (4)$$

we notice that the floor 0 appears twice and because the person travels from/to there in two separate trips, so when working with the floor numbers, we'll need to weight trips to/from the floor 0 at double the weight of the others. We can now calculate

$$E(|rest - X|) = \frac{rest}{5} + \frac{4}{5} \cdot \frac{1}{10} \sum_{X=1}^{10} |rest - X| \quad (5)$$

we take the derivative to find the optimum

$$\frac{dE}{drest} = \frac{1}{5} + \frac{4}{50} \sum_{X=1}^{10} \text{sgn}(rest - X) \quad (6)$$

and

$$\frac{4}{50} \sum_{X=1}^{10} \text{sgn}(rest - X) + \frac{1}{5} = 0 \quad (7)$$

Which is a pain to solve so we turn to tricks. If we assume there are k floors below the resting point, this means the expression inside the sum will be $+1$ k times, and -1 $10 - k - 1$ times. Therefore

$$D = \frac{4}{50} \sum_{X=1}^{10} \text{sgn}(rest - X) + \frac{1}{5} = \frac{1}{5} + \frac{4}{50} (k + k - 9) = \frac{10 - 36 + 8k}{50} = \frac{8k - 26}{50} \quad (8)$$

plugging in the different possible values for k

$$D(0) = -\frac{26}{50} \quad D(1) = -\frac{18}{50} \quad D(2) = -\frac{10}{50} \quad D(3) = -\frac{2}{50} \quad D(4) = \frac{6}{50} \quad (9)$$

so our two suspect floors are 3 and 4. We look at the values at the floors:

$$E(|3 - X|) = \frac{3}{5} + \frac{4}{50} \sum_{X=1}^{10} |3 - X| = \text{calculator} = \frac{77}{25} \quad (10)$$

$$E(|4 - X|) = \frac{4}{5} + \frac{4}{50} \sum_{X=1}^{10} |4 - X| = \text{calculator} = \frac{74}{25} \quad (11)$$

so floor 4 gives a lower expected value, meaning it is our optimal floor.

2 Peaches and Oranges

Since we have no information on any distributions or chances, we'll treat the fruit counts as being distributed poisson.

2.1 Oranges in the 2376 day

We can use the total number of peaches counted to find the expected value of peaches per day

$$\lambda = 10500/7 = 1500 \quad (12)$$

so the number of oranges is

$$N = 2376 - \lambda = 876 \quad (13)$$

The number of fruit counted the specific day is exact so has no variance. The number of peaches has variance which comes both from the variance on the actual count of peaches and from the daily count fluctuating around the average we calculate.

$$\sigma_{peach, week} = \sqrt{10500} \sim 102.5 \quad (14)$$

but since we divided it by 7 to look at each day

$$\sigma_{peach, day} = \frac{102.5}{7} \sim 14.6 \quad (15)$$

then the variance due to the actual daily number not being the exact average

$$\sigma_{day} = \sqrt{1500} \sim 38.7 \quad (16)$$

so the number of oranges that day is

$$N = 876 \pm 14.6 \pm 38.7 \quad (17)$$

and since those are independent the total error is

$$\sigma_{total} = \sqrt{(14.6)^2 + (38.7)^2} \sim 41.4 \quad (18)$$

so

$$N = 876 \pm 41.4 \quad (19)$$

2.2 Expected Daily Count

This time the count of total fruit is also uncertain. We treat it as a single measurement from a poisson distribution meaning

$$\lambda_{fruit} = 2376, \quad \sigma_{fruit} \sim 48.7 \quad (20)$$

and as for the count of peaches, this time we don't have the second source of uncertainty, since we don't need to consider whether the specific day was average or not, since we're looking at the average itself, so

$$\lambda_{peaches} = 1500, \quad \sigma_{peaches} = 14.6 \quad (21)$$

which gives us

$$N = 876 \pm 48.7 \pm 14.6 \tag{22}$$

and again summing independent errors

$$\sigma = \sqrt{48.7^2 + 14.6^2} \sim 50.8 \tag{23}$$

so

$$N = 876 \pm 50.8 \tag{24}$$

3 Resistors

Since the resistors are connected in parallel, we'll find the equivalent total resistance using

$$\frac{1}{R_{tot}} = \sum \frac{1}{R} \Rightarrow R_{tot} = \frac{R_1 R_2 R_3}{R_1 R_2 + R_1 R_3 + R_2 R_3} \quad (25)$$

giving us

$$R_{tot} = 3.3\text{k}\Omega \quad (26)$$

3.1 Total error

The error on the total resistance will have to be calculated using error propagation

$$\sigma^2 = \left(\frac{\partial R_{tot}}{\partial R_1} \right)^2 \sigma_{R1}^2 + \left(\frac{\partial R_{tot}}{\partial R_2} \right)^2 \sigma_{R2}^2 + \left(\frac{\partial R_{tot}}{\partial R_3} \right)^2 \sigma_{R3}^2 \quad (27)$$

$$= 3 \left(\frac{R_2 R_3 (R_1 R_2 + R_1 R_3 + R_2 R_3) - R_1 R_2 R_3 (R_2 + R_3)}{(R_1 R_2 + R_1 R_3 + R_2 R_3)^2} \right)^2 \sigma_{R1}^2 \quad (28)$$

$$= 3 \left(\frac{R_1 R_2^2 R_3 + R_1 R_2 R_3^2 + R_2^2 R_3^2 - R_1 R_2^2 R_3 - R_1 R_2 R_3^2}{(R_1 R_2 + R_1 R_3 + R_2 R_3)^2} \right)^2 \sigma_{R1}^2 \quad (29)$$

$$= 3 \left(\frac{R^4}{9R^4} \right)^2 \sigma_R^2 = \frac{1}{27} \sigma_R^2 \quad (30)$$

$$= \frac{1}{27} (0.05 \cdot 10\text{k}\Omega)^2 \quad (31)$$

$$= 9259\Omega^2 \quad (32)$$

so

$$\sigma = 96\Omega \quad (33)$$

where we used the fact the resistors are identical several times.

3.2 A single resistor

This time we only have a single source of error.

$$\sigma = 0.05 \cdot 3.3\text{k}\Omega = 165\Omega \quad (34)$$

3.3 in series instead

To get the same voltage we need the same resistance. The total resistance we found was $3.3\text{k}\Omega$ so we need three resistors each $1.1\text{k}\Omega$. The equation for total resistance is

$$R_{tot} = R_1 + R_2 + R_3 \quad (35)$$

so

$$\sigma^2 = 3 \left(\frac{\partial R_{tot}}{\partial R} \right)^2 \sigma_R^2 = 3\sigma_R^2 = 3 \cdot (0.05 \cdot 1.1\text{k}\Omega)^2 = 9075\Omega^2 \quad (36)$$

so

$$\sigma = 95\Omega \tag{37}$$

3.4 Different resistances vs the same

Yes there is a difference. Since they are independent elements, we sum the squares of the standard deviations then take the square root. If we look at the limit where one resistor has all the resistance and the others have resistance 0, the error is the same as the error on the individual resistor. Moving resistance to another resistor takes the value out from the square in the error calculation and moves it to a multiplication inside the root, which reduces it.

4 nonlinear system

The python code solving this with documentation is attached as several python files, which will be explained fully in appendix A. For all the tests I used the reduced chi-square test, which is just the chi-squared divided by the degrees of freedom, for convenience.

4.1 Representative measurements

The results from running the code for the representative measurements are

Model	χ^2	Reduced χ^2	p-value
Y_1	84.6	6.5	1.5×10^{-12}
Y_2	163.6	12.6	3.7×10^{-28}
Y_3	26.0	2.0	0.017

Table 1 – The chi-square, reduced chi-square, and p-value for the best fit for each model to the representative measurement dataset.

with the optimal parameters being

Model	A	B	C	D
Y_1	128.56	0.22	-186.57	-133.43
Y_2	-175.27	Inf.	0.4683	-67.60
Y_3	-1.5	28.04	-100.47	-159.35

Table 2 – Optimal parameters found using the python code. Note the infinity for B in model Y_2 , which results in the optimizer going to arbitrarily large values, when it's allowed to.

The benefits of each hypothesis is as follows:

- Y_1 a sum of two sine functions with double the frequency. From plotting the representative data, it indeed looks like the data might be some kind of oscillatory behavior, so the choice makes sense. If we expect the signal to be a sine wave with sine-like disturbances, this would capture it.
- Y_2 a decaying sine. The benefit of this is that it allows the ability to capture a signal whose amplitude decays over time. The problem is that if the system does not decay, this will be unable to capture the behavior.
- Y_3 a cubic polynomial. The benefits are that this is a much easier hypothesis to use for calculations and to test, and it fits the dataset in this part very well. The problem is that it assumes the behavior is of a cubic equation, which relies very strongly on an assumption the data will conform to that assumption beyond the measurements in the data.

The initial conclusion we can reach from this data is that models Y_1 and Y_2 can be ruled out for this dataset. Their high reduced chi-square combined with a miniscule p-value mean that the difference between the measurements and the model's prediction are both large and highly unlikely to be statistical errors. Y_3 also has a fairly high chi-square at 2 meaning the fit is still not great, but it's within workable limits.

4.2 Real Measurement

After looking at the raw data in a plot, I noticed that it can be broken up into two zones:

1. A background measurement
2. The signal region

and it appears that the noise in both regions is different. The signal measurement therefore itself introduces noise which we can't account for just with the background section. Looking at a histogram of the values in the background measurement I saw it looked fairly gaussian so I ran a chi-square test which resulted in

$$\chi^2/\text{dof} = 2.15, \quad \text{p value} = 0.018 \quad (38)$$

which is not a great result, but given I have no other information to work off of I'll accept an assumption the noise is gaussian in the background measurement with the following parameters.

$$\mu = -2.688 \pm 0.132, \quad \sigma = 3.68 \quad (39)$$

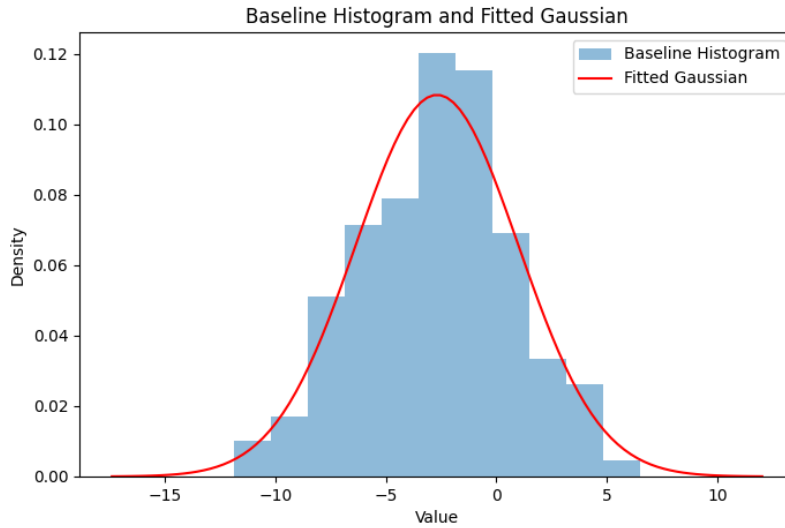


Figure 1 – Histogram of the values recorded in the section before the signal begins. A gaussian curve fit is presented on top.

A further analysis that could be made is whether the noise is somehow self-correlated, i.e that not all measurements in the noise really are random, which if removed could be characterized better by a gaussian, but this is out of scope. To analyze the noise in the signal region, I have to first isolate the signal. Since I don't know which of the hypothesis are correct and I don't want to rush to the next part, I isolated it by performing a rolling average of the values, with a window of 20 measurements. This window was significantly smaller than the oscillations in the signal so it did not skew the signal too much.

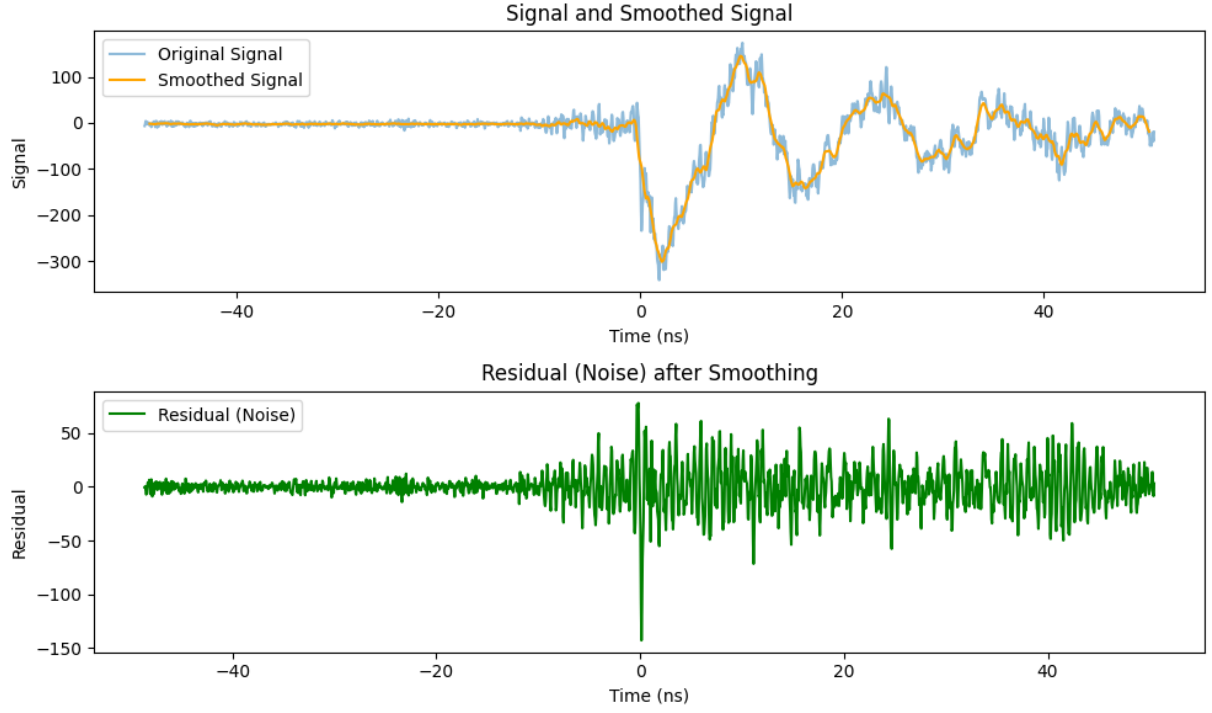


Figure 2 – Top panel: The raw data in blue and the smoothed out signal in yellow. Bottom panel: The residual, i.e the raw data with the smoothed out signal subtracted.

I then subtracted the cleaned signal from the raw data and found the noise in the signal region. After plotting a histogram of it, it also appeared quite gaussian, so I ran another chi-square test:

$$\chi^2/dof = 0.81, \quad p = 0.615 \quad (40)$$

which indicates that this time the noise is actually somewhat well described as gaussian:

$$\mu = -0.277, \quad \sigma = 22.438 \quad (41)$$

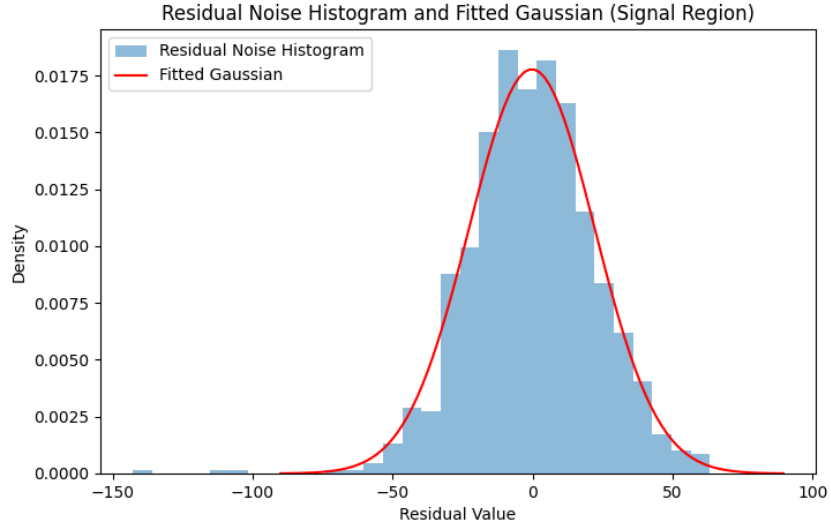


Figure 3 – Histogram of the values of the residual noise with a fitted gaussian.

next to characterize the signal I split the signal into bins of 0.5ns, and for each of those I calculated the mean, which is the total count N , then from it subtracted the baseline noise's mean which acts as B , giving me $S = N - B$. I then calculated the error from its own standard deviation divided by the root of the number of counts in the bin combined with the error on B .

$$S_{\min} = -295 \pm 7 \quad \text{at } t = 2.11\text{ns}, \quad S_{\max} = 154 \pm 4 \quad \text{at } t = 10.11\text{ns} \quad (42)$$

with significance

$$seg_{\min} = 42, \quad seg_{\max} = 36 \quad (43)$$

meaning the signal is highly significant in comparison to the noise.

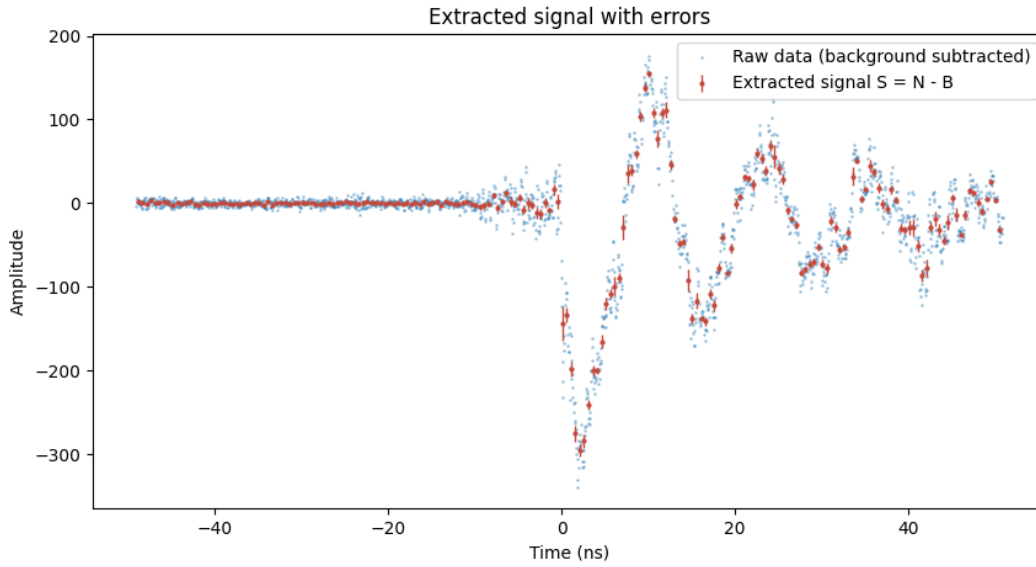


Figure 4 – The raw data in blue with the extracted signal, with errorbars, on top in red.

4.3 Simulation

In this part all that's needed is to load and properly process the data of the simulation then run the same tests as I did for the other three hypothesis. A note on the data processing, the simulation's times appeared different from the others, so I believed the times needed to be shifted. Instead of trying to manually find the right shift, I found the correct shift with the same optimization technique used for the other parameters in the question, with minimizing the chi-square between the simulation and the measured data. I decided to re-fit the hypothesis Y_i to the real measurement data from part b, to give each hypothesis its best shot. This resulted in the following

Hypothesis	χ^2/dof
Y_1	89.98
Y_2	27.92
Y_3	221.20
Simulation	118.03

Table 3 – The reduced chi squared for each hypothesis. The p value is not presented because it is so small it underflows the precision of floating point numbers, meaning it's basically 0 for ever fit.

Since the p value is effectively 0 for all of the fits, but none of the reduced chi-square tests give a good result, every hypothesis is rejected as a *good* fit. But, since we were asked for the best, rather than what's good, the best fit is the one with the lowest chi-squared, that being Y_2 . Y_3 is completely rejected because it can't capture the behavior of the data at all, and Y_1 is rejected because it's simply worse. The simulation too is rejected for this reason.

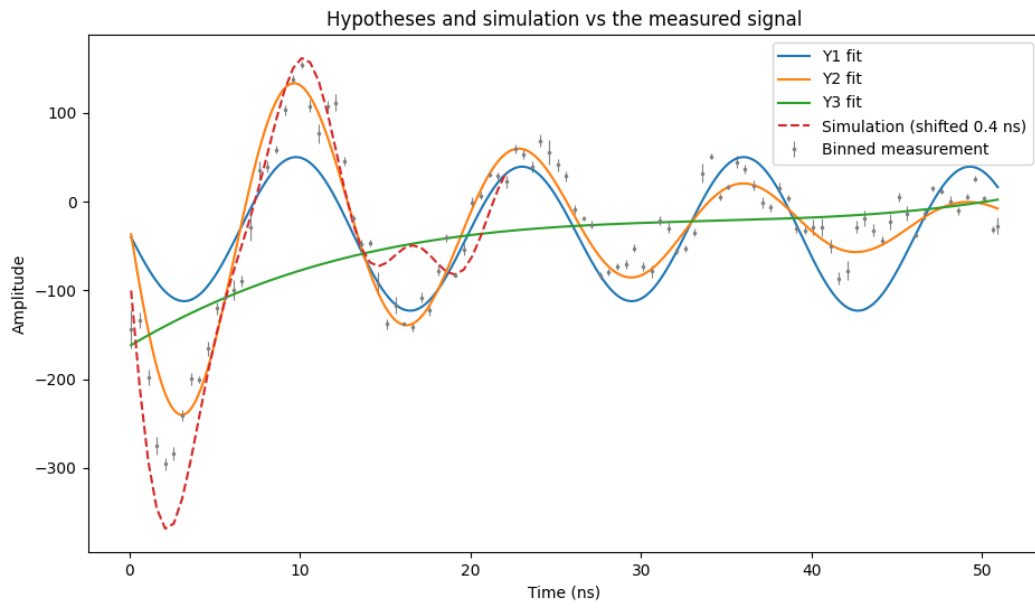


Figure 5 – The 3 models from part a and the simulation from part c (with the small shift found) plotted over the measurements from part b (after being binned).

A Question 4 Code Explanation

The code for this question is split into several files:

q4.py Wrapper for the other scripts. Originally everything was in this file, so to preserve my workflow I kept it, and it just calls the other files. It's completely optional.

q4_common.py Contains functions useful for all the sections of the question which are called multiple times.

q4_part_a.py The code for part a.

q4_part_b.py The code for part b.

q4_part_c.py The code for part c.

A.1 Common functions

The functions requiring an explanation are explained here. Some, like the excel loading function, chi squared calculations, etc, are simple enough that I don't think they need an explanation.

A.1.1 The models

Y_1 , Y_2 , and Y_3 are all defined here for use everywhere.

A.1.2 model_slope

To perform a fit on a model which also has errors on the time axis, we need to propagate the error, which itself needs the derivative of the model tested w.r.t time. This calculates this derivative. Can't use a built in gradient function because we're taking the derivative of a python function. The derivative is calculated using the derivative definition, with a small step chosen arbitrarily.

A.1.3 check_fit_quality

Calculates the number of degrees of freedom, reduced chi squared (chi squared divided by the degrees of freedom), and the p value.

A.1.4 solve_linear

Performs a weighted least squares fit for a model that is linear: $y(t) = \sum_j p_j f_j(t)$. Minimizing χ^2 in this case has a direct solution: the weighted normal equations

$$(X^T W X) \mathbf{p} = X^T W \mathbf{y}, \quad W = \text{diag}(1/\sigma_i^2),$$

where $X_{ij} = f_j(t_i)$ is the matrix of basis functions at the data points. The function builds X from the basis columns and solves this system without minimization, and points with smaller errors get more weight. Used to solve Y_3 and the linear parameters of Y_1 and Y_2 .

A.1.5 nonlinear_ranges

Creates the ranges used to limit the scanning for values of the parameters. Needed because for Y_2 B sometimes can diverge.

A.1.6 profile_params

Function meant to be used only for the nonlinear models, Y_1 and Y_2 . Takes the linear part of each model and throws it to the linear solver. Returns the model with the linear parameters solved.

A.1.7 fit_model

performs the full fit for the model to the given points, assuming no error on the time. Y_3 is sent to be solved directly, other models have the linear parameters solved and the nonlinear optimized. For the nonlinear parameters, the following algorithm is performed:

1. A D -dimensional grid of the possible values for all the different parameters is created to scan over. The grid is then flattened into a 1D array to simplify iteration (theoretically speaking, this could be replaced with a multi-dimensional optimization algorithm such as stochastic gradient descent, but it's overkill for this).
2. for each point in the flattened grid, the chi squared is calculated
3. if the chi squared is the best seen so far, save the values at that point as the best
4. at each step, zoom in on the best nonlinear parameters found so far. This helps convergence, assuming the chi-squared landscape is mostly monotonous.
5. return the best parameters found

A.1.8 fit_with_t_errors

If there are errors on the time, the above algorithm is insufficient, since it doesn't take the time errors into account. The total error used for most of the calculations needs to have the time error propagated. This function does:

1. Fit the model assuming no time error
2. iterate *max_iter* times or until the chi squared calculated with the updated total error changes very little:
 - (a) propagate the time error
 - (b) perform the fit with the new error
 - (c) if chi squared doesn't change a lot, fit is declared good
 - (d) update the parameters and chi squared to the ones calculated in the loop
3. return the fitted parameters, chi squared, and total error

A.2 Part a

This file contains only a single function because its logic is implemented almost entirely in the common functions file. The function loads the data, then performs the fit with time errors included, gets the reduced chi squared and p value, and prints a table with all the parameters and statistical values.

A.3 Part b

A.3.1 `check_background_gaussianity`

Checks if a given signal (since this was made for the noise it's named this way) is gaussian noise.

1. Calculates the mean and standard deviation
2. Creates a histogram of the values in the given signal (to simplify plotting the bins are limited to $\pm 2.5\sigma$ with everything beyond in a single bin per side)
3. performs a chi-squared test comparing the signal's histogram to a gaussian
4. plots the comparison and returns the statistical values

A.3.2 `get_signal`

Since the noise before the signal and during the signal wasn't different, the way I thought of to isolate the signal was to use a rolling window average. The function uses a constant convolution kernel to smooth out the signal. Plots the cleaned signal and the residual noise. Returns the signal, noise, and other unused parameters.

A.3.3 `noise_during_signal`

Isolates the noise during the signal using `get_signal`, checks if it's gaussian, calculates statistics for it, and plots it.

A.3.4 `extract_signal`

This function gets the signal from the raw data by subtracting the baseline noise, and defines the errors on it. To get the errors on the signal, we need to consider the noise during the signal, and to do that we have to look at bins of datapoints of the signal, rather than point by point. The signal is split into bins over time, then the signal is defined as the mean of the values in the bin, and the error is calculated from the spread of the values of the signal within that bin.

A.3.5 `part_b`

The main function of this part finds the background using the measurement time before the signal begins and performs the analysis on it, then estimates the signal and its error using the above function. It then prints everything.

A.4 Part c

A.4.1 `align_simulation`

When I first checked the data here and compared it to the previous parts, I noticed that the times given here are significantly different from the other parts. I suspected this would make it hard to perform the fit of the simulation model to the experiment data, so this function which aligns the data was made. Instead of trying to find the right offset by hand, this function finds the right offset by fitting the simulation to the experiment data. Since the simulation is just a set of points, the only variable that can be altered in it is the time offset, so there's a single parameter to optimize for. The function performs this optimization very similarly to the previous optimizations.

1. Fix the resolution mismatch. The simulation has many more data points than the experiment data, and trying to fit them against each other at this point will cause issues. The simulation data is resampled to have the same number of points.
2. Loop over all the possible time shift values, and for each
 - (a) clip the data to only the region where there is an overlap between the simulation and measurement
 - (b) perform the fit and chi-squared test between the simulation and measurement
 - (c) if this fit is the best, remember it
3. return the best fit found

Eventually it turned out, after running part c, that the offset was extremely small so it made almost no difference, but this was kept because it gives the simulation its best chance.

A.4.2 part_c

The function fits all models (Y_1 through Y_3 and the simulation) against the experimental data, then prints a table comparing them all, and plots all of them together.